



Ramificación y poda (*branch & bound*)

Simona Bernardi

Universidad de Zaragoza

Presentación adaptada de la original de J. Campos

Este documento está sujeto a una licencia de uso Creative Commons. No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.





Resumen

- Método general
- El juego del 15
- Problemas de optimización
 - Un problema de planificación de tareas a plazo fijo
 - El problema de la mochila 0-1
 - El problema del viajante de comercio



Ramificación

- Al igual que el método de búsqueda con retroceso:
 - Las soluciones del problema tienen que satisfacer a ciertas condiciones
 - Se genera el espacio de soluciones, organizándolo en un árbol (en general, en un grafo)
 - No se genera el árbol completo, sino que se podan bastantes ramas



Ramificación

- Al igual que el método de búsqueda con retroceso:
 - Las soluciones del problema tienen que satisfacer a ciertas condiciones
 - Se genera el espacio de soluciones, organizándolo en un árbol (en general, en un grafo)
 - No se genera el árbol completo, sino que se podan bastantes ramas

Terminología

- Nodo **vivo**: nodo del espacio de soluciones del que no se han generado aún todos sus hijos
- Nodo **muerto**: nodo del que no se van a generar más hijos porque
 - no hay más
 - no es completable, es decir viola las restricciones
 - no producirá una solución mejor que la solución actual
- Nodo **en expansión**: nodo del que se están generando hijos

Diferencia con el método de búsqueda con retroceso



- Búsqueda con retroceso: tan pronto como se genera un nuevo hijo del nodo en expansión, este hijo pasa a ser el nodo en curso
⇒ búsqueda en profundidad
- Ramificación y poda: se generan todos los hijos del nodo en expansión antes de que **cualquier otro nodo vivo** pase a ser el nuevo nodo en expansión



Diferencia con el método de búsqueda con retroceso

- Búsqueda con retroceso: tan pronto como se genera un nuevo hijo del nodo en expansión, este hijo pasa a ser el nodo en curso
⇒ búsqueda en profundidad
- Ramificación y poda: se generan todos los hijos del nodo en expansión antes de que **cualquier otro nodo vivo** pase a ser el nuevo nodo en expansión

Consecuencia

- Búsqueda con retroceso: los únicos nodos vivos son los que están en el camino de la raíz al nodo en expansión
- Ramificación y poda: puede haber más nodos vivos. Se necesita una estructura de datos auxiliar para almacenar **la lista de nodos vivos**



Recorrido del árbol de soluciones

Diferentes estrategias de elegir el siguiente nodo de la lista de nodos vivos



Distintos órdenes de recorrido del árbol de soluciones



Recorrido del árbol de soluciones

Diferentes estrategias de elegir el siguiente nodo de la lista de nodos vivos



Distintos órdenes de recorrido del árbol de soluciones

- **FIFO**: la lista de nodos vivos es una cola $\Rightarrow$ recorrido por niveles
- **LIFO**: la lista de nodos vivos es una pila $\Rightarrow$ *D-search*



Recorrido del árbol de soluciones

Diferentes estrategias de elegir el siguiente nodo de la lista de nodos vivos



Distintos órdenes de recorrido del árbol de soluciones

- **FIFO**: la lista de nodos vivos es una cola $\Rightarrow$ recorrido por niveles
- **LIFO**: la lista de nodos vivos es una pila $\Rightarrow D$ -search
- **Mínimo coste**: la lista de nodos vivos es una cola con prioridades $\Rightarrow$ recorrido extraño
- La prioridad de un nodo se calcula con una **función de estimación** que mide cuánto de *prometedor* es un nodo



Punto clave: función de estimación

Encontrar un buen orden de recorrido (o ramificación) de los nodos



Definir una buena función de prioridad de los nodos vivos, para que las soluciones buenas se encuentren rápidamente

El juego del 15

- El problema original

1	2	3	4
5	6	7	8
9	10	11	12
13	15	14	



El juego del 15

- El problema original

1	2	3	4
5	6	7	8
9	10	11	12
13	15	14	

1	2	3	4
5	6	7	8
9	10	11	
13	15	14	12



El juego del 15

- El problema original

1	2	3	4
5	6	7	8
9	10	11	12
13	15	14	

1	2	3	4
5	6	7	8
9	10	11	
13	15	14	12

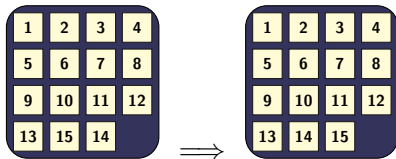
1	2	3	4
5	6	7	8
9	10		11
13	15	14	12





El juego del 15

- El problema original



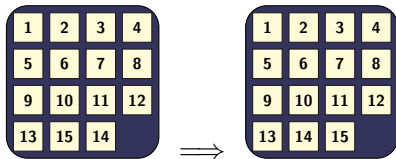
Samuel Loyd and the 15 puzzle

<https://mathematicalmysterytour.blogspot.com/2014/10/sam-loyd-and-15-puzzle.html>



El juego del 15

- El problema original



Samuel Loyd and the 15 puzzle

<https://mathematicalmysterytour.blogspot.com/2014/10/sam-loyd-and-15-puzzle.html>

- La solución de fuerza bruta:
 - Buscar en el espacio de estados (todas las configuraciones alcanzables desde la configuración inicial) hasta encontrar el objetivo
 - Hay $16! \approx 20,9 \times 10^{12}$ configuraciones, aunque **sólo (?) la mitad pueden alcanzarse** desde la configuración inicial propuesta por Loyd

Configuraciones alcanzables

- Numeremos las casillas de 1 a 16





Configuraciones alcanzables

- Numeremos las casillas de 1 a 16
- Dada una configuración:
 - Sea $1 \leq pos(i) \leq 15$ la posición de la ficha número i , (vacía, $pos(16)$)
 - Para cada ficha i , sea $m(i)$ el número de fichas $j < i$ t.q. $pos(j) > pos(i)$

1	2	3	4
5	6		8
9	10	7	11
13	14	15	12

$m(i) = 0$, para $i = 1, 2, 3, 4, 5, 6, 7$

$m(8) = m(9) = m(10) = 1$

$m(11) = 0$

$m(12) = 0$

$m(13) = m(14) = m(15) = 1$

$m(16) = 9$



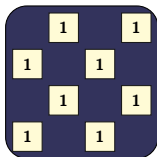
Configuraciones alcanzables

- Numeremos las casillas de 1 a 16
- Dada una configuración:
 - Sea $1 \leq pos(i) \leq 15$ la posición de la ficha número i , (vacía, $pos(16)$)
 - Para cada ficha i , sea $m(i)$ el número de fichas $j < i$ t.q. $pos(j) > pos(i)$



$m(i) = 0$, para $i = 1, 2, 3, 4, 5, 6, 7$
 $m(8) = m(9) = m(10) = 1$
 $m(11) = 0$
 $m(12) = 0$
 $m(13) = m(14) = m(15) = 1$
 $m(16) = 9$

- Sea $x = 1$ si la casilla vacía está en amarillo, $x = 0$ en otro caso





Configuraciones alcanzables

Teorema

La configuración objetivo es alcanzable desde una cierta configuración inicial si para esta configuración:

$$\sum_{i=1}^{16} m(i) + x \quad \text{es par.}$$



Configuraciones alcanzables

Teorema

La configuración objetivo es alcanzable desde una cierta configuración inicial si para esta configuración:

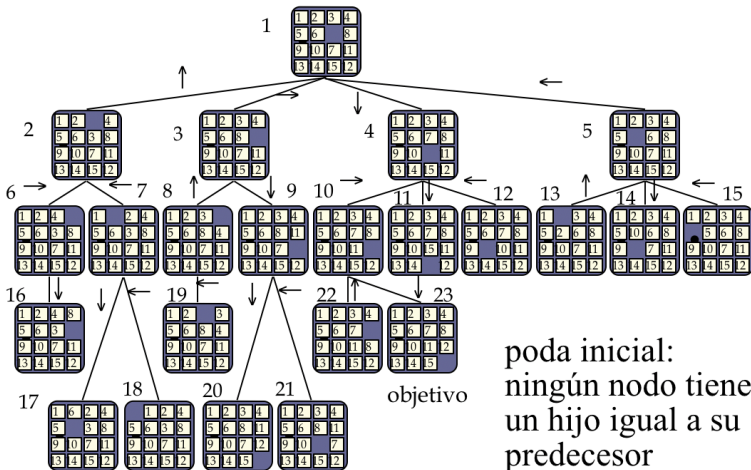
$$\sum_{i=1}^{16} m(i) + x \quad \text{es par.}$$

- Para el ejemplo anterior, dicha suma vale 16: la configuración objetivo es alcanzable
- En el problema original de Loyd, la configuración objetivo no es alcanzable



Búsqueda en anchura

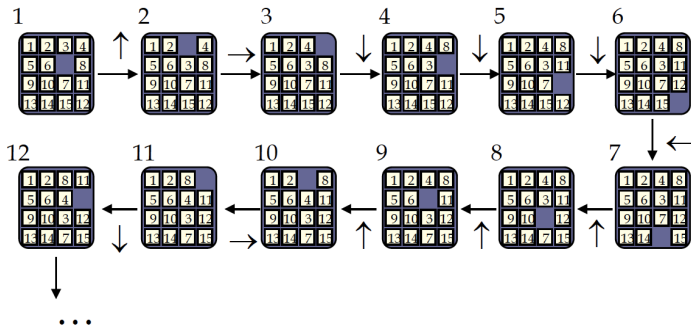
- Hijos de cada nodo = movimientos de la casilla vacía (arriba,dcha,abajo,izda)





Búsqueda en profundidad

- Orden de los movimientos (arriba,dcha,abajo,izda)



- Sigue la rama izquierda del árbol



Estrategias de búsqueda

- Estrategias a **ciegas**:
 - FIFO: recorrido en anchura o por niveles
 - Relativamente útil si el nodo solución está cerca de la raíz
 - LIFO: $\approx$ recorrido en profundidad (también llamado *D-search* en el libro de Horowitz et al.)
 - Puede no encontrar la solución (problemas con los ciclos)



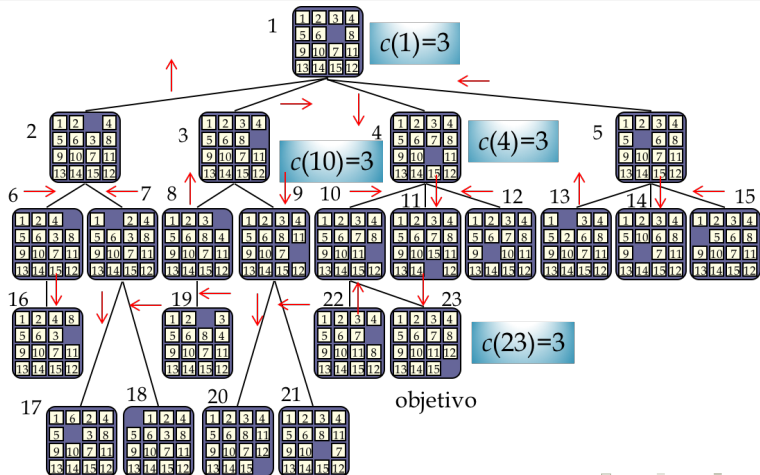
Estrategias de búsqueda

- Estrategias a **ciegas**:
 - FIFO: recorrido en anchura o por niveles
 - Relativamente útil si el nodo solución está cerca de la raíz
 - LIFO: $\approx$ recorrido en profundidad (también llamado *D-search* en el libro de Horowitz et al.)
 - Puede no encontrar la solución (problemas con los ciclos)
- Estrategias no ciegas (*informadas*, en inteligencia artificial)
 - Buscan la solución adaptando el camino de búsqueda en el árbol de soluciones a la instancia del problema a solucionar
 - Se necesita definir una **función de coste** para los nodos del árbol



Ejemplo de función de coste

$c(x)$ = longitud desde la raíz hasta el nodo objetivo más cercano que es descendiente de x





Función de estimación

- Problema: el cálculo de la función c
 - Un ejemplo de problemas en los que es posible: aquéllos en los que hay soluciones voraces



Función de estimación

- Problema: el cálculo de la función c
 - Un ejemplo de problemas en los que es posible: aquéllos en los que hay soluciones voraces

Función de estimación (heurística)

$$\hat{c}(x) = f(x) + \hat{g}(x)$$

$f(x)$ = longitud del camino de la raíz a x

$\hat{g}(x)$ = estimación de la longitud del camino de x hasta la solución más cercana



Función de estimación

- Problema: el cálculo de la función c
 - Un ejemplo de problemas en los que es posible: aquéllos en los que hay soluciones voraces

Función de estimación (heurística)

$$\hat{c}(x) = f(x) + \hat{g}(x)$$

$f(x)$ = longitud del camino de la raíz a x

$\hat{g}(x)$ = estimación de la longitud del camino de x hasta la solución más cercana

Propiedades de $\hat{c}$:

- Fácil de calcular
- Si x es una hoja o solución: $c(x) = \hat{c}(x)$



Estrategia de mínimo coste

$\hat{g}(x)$ = número de casillas no vacías que no están en su sitio objetivo ($\hat{c}(x) \leq c(x), \forall x$)

- La lista de nodos vivos es una cola de nodos x con prioridades $\hat{c}(x)$



Estrategia de mínimo coste

$\hat{g}(x)$ = número de casillas no vacías que no están en su sitio objetivo ($\hat{c}(x) \leq c(x), \forall x$)

- La lista de nodos vivos es una cola de nodos x con prioridades $\hat{c}(x)$

- Nodo inicial

1

1	2	3	4
5	6		8
9	10	7	11
13	14	15	12

 $\hat{c}(1) = 0 + 3$



Estrategia de mínimo coste

$\hat{g}(x)$ = número de casillas no vacías que no están en su sitio objetivo ($\hat{c}(x) \leq c(x), \forall x$)

- La lista de nodos vivos es una cola de nodos x con prioridades $\hat{c}(x)$

- Nodo inicial

1

1	2	3	4
5	6		8
9	10	7	11
12	13	14	15

 $\hat{c}(1) = 0 + 3$

- Se generan sus hijos y se añaden a la cola

2

1	2		4
5	6	3	8
9	10	7	11
12	13	14	15

 $\hat{c}(2) = 1 + 4$

3

1	2	3	4
5	6	8	
9	10	7	11
12	13	14	15

 $\hat{c}(3) = 1 + 4$

4

1	2	3	4
5	6	7	8
9	10		11
12	13	14	15

 $\hat{c}(4) = 1 + 2$

5

1	2	3	4
5		6	8
9	10	7	11
12	13	14	15

 $\hat{c}(5) = 1 + 4$



Estrategia de mínimo coste

- Se elige el mínimo (4), se elimina de la cola y se generan sus hijos

2

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

$$\hat{c}(2) = 1 + 4$$

3

1	2	3	4
5	6	8	
9	10	7	11
13	14	15	12

$$\hat{c}(3) = 1 + 4$$

5

1	2	3	4
5		6	8
9	10	7	11
13	14	15	12

$$\hat{c}(5) = 1 + 4$$

10

1	2	3	4
5	6	7	8
9	10	11	
13	14	15	12

$$\hat{c}(10) = 2 + 1$$

11

1	2	3	4
5	6	7	8
9	10	11	12
13	14		

$$\hat{c}(11) = 2 + 3$$

12

1	2	3	4
5	6	7	8
9		10	11
13	14	15	12

$$\hat{c}(12) = 2 + 3$$



Estrategia de mínimo coste

- Se elige el mínimo (4), se elimina de la cola y se generan sus hijos

2

1	2	3	4
5	6	8	8
9	10	7	11
12	11	12	12

$$\hat{c}(2) = 1 + 4$$

3

1	2	3	4
5	6	8	8
9	10	7	11
12	11	12	12

$$\hat{c}(3) = 1 + 4$$

5

1	2	3	4
5	6	8	8
9	10	7	11
12	11	12	12

$$\hat{c}(5) = 1 + 4$$

10

1	2	3	4
5	6	7	8
9	10	11	11
12	11	12	12

$$\hat{c}(10) = 2 + 1$$

11

1	2	3	4
5	6	7	8
9	10	11	11
12	11	12	12

$$\hat{c}(11) = 2 + 3$$

12

1	2	3	4
5	6	7	8
9	10	11	11
12	11	12	12

$$\hat{c}(12) = 2 + 3$$

- Se elige el mínimo (10), se elimina de la cola y se generan sus hijos

2

1	2	3	4
5	6	8	8
9	10	7	11
12	11	12	12

$$\hat{c}(2) = 1 + 4$$

3

1	2	3	4
5	6	8	8
9	10	7	11
12	11	12	12

$$\hat{c}(3) = 1 + 4$$

5

1	2	3	4
5	6	8	8
9	10	7	11
12	11	12	12

$$\hat{c}(5) = 1 + 4$$

11

1	2	3	4
5	6	7	8
9	10	11	11
12	11	12	12

$$\hat{c}(11) = 2 + 3$$

12

1	2	3	4
5	6	7	8
9	10	11	11
12	11	12	12

$$\hat{c}(12) = 2 + 3$$

22

1	2	3	4
5	6	7	8
9	10	11	8
12	11	12	12

$$\hat{c}(22) = 3 + 2$$

23

1	2	3	4
5	6	7	8
9	10	11	12
12	11	12	12

$$\hat{c}(23) = 3 + 0$$



Algoritmo de búsqueda mínimo coste

```
1 // Pre: raíz es el nodo inicial del árbol de búsqueda
2 // Post: escribe una solución de mínimo coste (si no hay, avisa)
3 void mínimoCoste(tipoNodo raíz) {
4     if esSolución(raíz) escribir(raíz);
5     else {
6         cola = crearColaVacía(); // cola con prioridades de nodos vivos
7         añadir(cola,<raíz,coste(raíz)>); éxito = false;
8         while not éxito and not esVacía(cola) {
9             nodo_E = seleccionar_min(cola); // elegir el nodo de mínimo coste estimado
10            eliminar(cola,nodo_E); // elimina el nodo de la cola
11            hijos = generaHijos(nodo_E); // genera todos sus hijos
12            while not éxito and not esVacío(hijos) {
13                nodo = extraerHijo(hijos);
14                if esSolución(nodo) { escribir(nodo); éxito = true; }
15                else { añadir(cola,<nodo,coste(nodo)>); }
16            }
17        }
18        if not éxito escribir("No hay solución");
19    }
20 }
```



Observaciones

- Si se desea escribir todo el camino desde la raíz hasta la solución, cada vez que se añade un nodo a la cola, hay que guardar en una **tabla auxiliar de padres de nodos** ese nodo junto a su padre



Observaciones

- Si se desea escribir todo el camino desde la raíz hasta la solución, cada vez que se añade un nodo a la cola, hay que guardar en una **tabla auxiliar de padres de nodos** ese nodo junto a su padre
- La terminación del algoritmo sólo está garantizada si el espacio de estados es finito



Observaciones

- Si se desea escribir todo el camino desde la raíz hasta la solución, cada vez que se añade un nodo a la cola, hay que guardar en una **tabla auxiliar de padres de nodos** ese nodo junto a su padre
- La terminación del algoritmo sólo está garantizada si el espacio de estados es finito
- Si el espacio de estados es infinito y existe al menos una solución, se puede garantizar la terminación con una elección adecuada de la función de estimación



Observaciones

- Si se desea escribir todo el camino desde la raíz hasta la solución, cada vez que se añade un nodo a la cola, hay que guardar en una **tabla auxiliar de padres de nodos** ese nodo junto a su padre
- La terminación del algoritmo sólo está garantizada si el espacio de estados es finito
- Si el espacio de estados es infinito y existe al menos una solución, se puede garantizar la terminación con una elección adecuada de la función de estimación
- Si el espacio de estados es infinito y no hay solución, el algoritmo no termina...
 - Se suele restringir la búsqueda a nodos con coste estimado menor que C (cota)



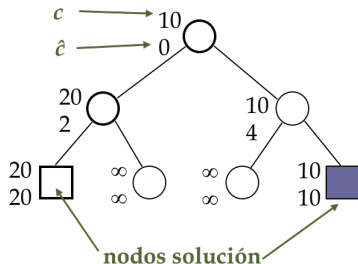
Problemas de optimización

- La función o coste a minimizar ya no es la distancia de la raíz a la solución, sino una función objetivo dada
- ¿Qué ocurre si interesa buscar una solución óptima (es decir de coste mínimo)?
- ¿Encuentra el algoritmo mínimoCoste esa solución?



Problemas de optimización

- La función o coste a minimizar ya no es la distancia de la raíz a la solución, sino una función objetivo dada
- ¿Qué ocurre si interesa buscar una solución óptima (es decir de coste mínimo)?
- ¿Encuentra el algoritmo mínimoCoste esa solución? **No**

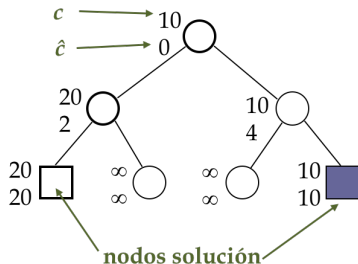


- Se expande la raíz, sus hijos se añaden a la cola de nodos vivos
- Se elige el hijo izquierdo, se expande y se obtiene una solución factible de coste 20
- La solución óptima tiene coste 10.
- ¿Porqué?



Problemas de optimización

- La función o coste a minimizar ya no es la distancia de la raíz a la solución, sino una función objetivo dada
- ¿Qué ocurre si interesa buscar una solución óptima (es decir de coste mínimo)?
- ¿Encuentra el algoritmo mínimoCoste esa solución? **No**



- Se expande la raíz, sus hijos se añaden a la cola de nodos vivos
- Se elige el hijo izquierdo, se expande y se obtiene una solución factible de coste 20
- La solución óptima tiene coste 10.
- ¿Porqué?

$$\exists x, y : \hat{c}(x) < \hat{c}(y) \wedge c(x) > c(y)$$



Problemas de optimización

Teorema

Si para cada nodo x del árbol del espacio de estados $\hat{c}(x)$ es una estimación de $c(x)$ tal que para todo par de nodos y, z :

$$\hat{c}(y) < \hat{c}(z) \Leftrightarrow c(y) < c(z)$$

entonces el algoritmo `mínimoCoste` termina y encuentra siempre una solución óptima.



Problemas de optimización

Teorema

Si para cada nodo x del árbol del espacio de estados $\hat{c}(x)$ es una estimación de $c(x)$ tal que para todo par de nodos y, z :

$$\hat{c}(y) < \hat{c}(z) \Leftrightarrow c(y) < c(z)$$

entonces el algoritmo `mínimoCoste` termina y encuentra siempre una solución óptima.

- **Problema:** no es fácil encontrar una función de estimación con tales características y fácil de calcular
- **Conformarse con:** una función $\hat{c}(x)$ fácil de calcular y tal que para todo nodo x : $\hat{c}(x) \leq c(x)$ y para cada solución: $\hat{c}(x) = c(x)$. En este caso, `mínimoCoste` no siempre encuentra una solución óptima, pero se puede modificar ligeramente...



Algoritmo de búsqueda mínimo coste modificado

```
1 // Pre: raíz es el nodo inicial del árbol de búsqueda
2 // Post: escribe una solución de mínimo coste (si no hay, avisa)
3 void mínimoCoste_V2(tipoNodo raíz) {
4     cola = crearColaVacía(); // cola con prioridades de nodos vivos
5     añadir(cola, <raíz, coste(raíz)>); éxito = false;
6     while not éxito and not esVacía(cola) {
7         nodo_E = seleccionar_min(cola); // elegir nodo en expansión
8         eliminar(cola, nodo_E);
9         if esSolución(nodo_E) { escribir(nodo_E); éxito = true; }
10        else {
11            hijos = generaHijos(nodo_E); // genera todos los hijos
12            for nodo ∈ hijos { // y los añade a la cola con prioridad
13                añadir(cola, <nodo, coste(nodo)>);
14            }
15        }
16    }
17    if not éxito escribir("No hay solución");
18 }
```

Diferencia entre los dos algoritmo de coste mínimo



- `mínimoCoste` si encuentra un hijo que sí es solución ya no incluye el resto de los hijos en la cola de nodos vivos con prioridades;
- `mínimoCoste_V2` incluye en la cola de nodos vivos con prioridades todos los hijos, sin chequear si son solución. ($\Leftarrow$ algoritmo más prudente)



Diferencia entre los dos algoritmo de coste mínimo

- `mínimoCoste` si encuentra un hijo que sí es solución ya no incluye el resto de los hijos en la cola de nodos vivos con prioridades;
- `mínimoCoste_V2` incluye en la cola de nodos vivos con prioridades todos los hijos, sin chequear si son solución. ($\Leftarrow$ algoritmo más prudente)

Teorema

Si para cada nodo x del árbol del espacio de estados $\hat{c}(x)$ es una función de estimación tal que:

$$\hat{c}(x) \leq c(x)$$

y para cada nodo solución x se verifica: $\hat{c}(x) = c(x)$, entonces si el algoritmo `mínimoCoste_V2` encuentra una solución, ésta es óptima.



Demostración teorema

Si se encuentra la solución `nodo_E`:

- ① `nodo_E` es el nodo en expansión: $\hat{c}(\text{nodo_E}) \leq \hat{c}(x) \quad \forall x \in \text{cola}$
- ② `nodo_E` es una solución: $\hat{c}(\text{nodo_E}) = c(\text{nodo_E})$
- ③ La función de estimación subestima el coste: $\hat{c}(x) \leq c(x) \quad \forall x \in \text{cola}$
- ④ También $c(x) \leq c(y)$ para cada nodo x del árbol, donde y es hijo de x
- ⑤ Luego $c(\text{nodo_E}) \leq c(x)$ para cada nodo x

Conclusión: `nodo_E` es una solución de mínimo coste (óptima).



Poda

- Dado el problema de minimización de una **función** $c(x)$
- Se utiliza una función de estimación $\hat{c}(x)$ tal que $\hat{c}(x) \leq c(x)$
- Suponer que se conoce una cota superior del valor mínimo de c :

$$c(x^*) = \min_x c(x) \leq \mathcal{U} \quad (\text{cota superior})$$

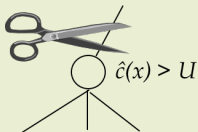


Poda

- Dado el problema de minimización de una **función** $c(x)$
- Se utiliza una función de estimación $\hat{c}(x)$ tal que $\hat{c}(x) \leq c(x)$
- Suponer que se conoce una cota superior del valor mínimo de c :

$$c(x^*) = \min_x c(x) \leq \mathcal{U} \quad (\text{cota superior})$$

Regla de poda



- Si $\hat{c}(x) > \mathcal{U}$, entonces x puede ser podado porque $\forall y$ solución descendiente de x :

$$c(y) \geq c(x) \geq \hat{c}(x) > \mathcal{U}$$

Poda



- Valor inicial de $\mathcal{U}$:
 - Utilizar alguna heurística (basada en información extra sobre el problema)
 - ∞



Poda

- Valor inicial de $\mathcal{U}$:
 - Utilizar alguna heurística (basada en información extra sobre el problema)
 - ∞
- Si al alcanzar solución x : $c(x) < \mathcal{U}$, se actualiza el valor de $\mathcal{U}$



Poda

- Valor inicial de $\mathcal{U}$:
 - Utilizar alguna heurística (basada en información extra sobre el problema)
 - ∞
- Si al alcanzar solución x : $c(x) < \mathcal{U}$, se actualiza el valor de $\mathcal{U}$
- Todos los nodos vivos de coste estimado mayor que $\mathcal{U}$ también se pueden podar



Construcción del algoritmo

Definición de la función de coste $c(x)$ de forma que $c(x)$ sea mínimo para todos los nodos que representan una solución óptima. ¿Cómo?



Construcción del algoritmo

Definición de la función de coste $c(x)$ de forma que $c(x)$ sea mínimo para todos los nodos que representan una solución óptima. ¿Cómo?

- Si x es solución factible del problema: $c(x) = F_{\text{objetivo}}(x)$



Construcción del algoritmo

Definición de la función de coste $c(x)$ de forma que $c(x)$ sea mínimo para todos los nodos que representan una solución óptima. ¿Cómo?

- Si x es solución factible del problema: $c(x) = F_{\text{objetivo}}(x)$
- Si x no es factible: $c(x) = \infty$



Construcción del algoritmo

Definición de la función de coste $c(x)$ de forma que $c(x)$ sea mínimo para todos los nodos que representan una solución óptima. ¿Cómo?

- Si x es solución factible del problema: $c(x) = F_{\text{objetivo}}(x)$
- Si x no es factible: $c(x) = \infty$
- Si x es solución parcial (x puede ser completada a factible):

$$c(x) = \min \{c(y) \mid y \text{ es descendiente de } x\}$$



Construcción del algoritmo

Definición de la función de coste $c(x)$ de forma que $c(x)$ sea mínimo para todos los nodos que representan una solución óptima. ¿Cómo?

- Si x es solución factible del problema: $c(x) = F_{\text{objetivo}}(x)$
- Si x no es factible: $c(x) = \infty$
- Si x es solución parcial (x puede ser completada a factible):

$$c(x) = \min \{c(y) \mid y \text{ es descendiente de } x\}$$

Tan difícil de calcular como resolver el problema original!



Construcción del algoritmo

Definición de la función de coste $c(x)$ de forma que $c(x)$ sea mínimo para todos los nodos que representan una solución óptima. ¿Cómo?

- Si x es solución factible del problema: $c(x) = F_{\text{objetivo}}(x)$
- Si x no es factible: $c(x) = \infty$
- Si x es solución parcial (x puede ser completada a factible):

$$c(x) = \min \{c(y) \mid y \text{ es descendiente de } x\}$$

Tan difícil de calcular como resolver el problema original!

- Se utiliza $\hat{c}(x)$ tal que $\hat{c}(x) \leq c(x)$ para todo x
- La función de estimación **estima el valor de la función objetivo y no el coste computacional** de alcanzar la solución



Puntos clave de método

- 1 Encontrar un buen orden de recorrido (o ramificación) de los nodos, es decir, definir una buena función de prioridad $\hat{c}$ de los nodos vivos, para que las soluciones buenas se encuentren rápidamente.
- 2 **Encontrar una buena función de poda, $\mathcal{U}$ para que se produzca el retroceso lo antes posible.**



Algoritmo de búsqueda ramificación y poda

```
1 // Pre: existe al menos un nodo de mínimo coste, existe una función  $\hat{c}$ 
2 //       y un valor inicial U:  $\hat{c}(x) \leq c(x) \leq U$ .
3 // Post: escribe el coste asociado a una solución óptima (mínimo coste)
4 void RamPodaMinCoste(tipoNodo raíz, double U) {
5     cola = creaColaVacía(); // cola con prioridades de nodos vivos
6     añadir(cola, <raíz,  $\hat{c}$ (raíz)>);
7     while not esVacía(cola) {
8         nodoE = seleccionar_min(cola); // elegir nodo en expansión
9         eliminar(cola, nodo_E);
10        for nodo  $\in$  hijos(nodo_E) {
11            if  $\hat{c}$ (nodo)  $\leq$  U { // poda basada en la cota
12                añadir(cola, <nodo,  $\hat{c}$ (nodo)>);
13                if esSolución(nodo) and  $c$ (nodo) < U { U =  $c$ (nodo); }
14            }
15        }
16        if esVacía(cola) or  $\hat{c}$ (seleccionar_min(cola))  $\geq$  U {
17            escribir(U); crearVacía(cola);
18        }
19    }
20 }
```



Observaciones

- Hay que mantener una variable auxiliar con el valor del último nodo solución que ha permitido actualizar U
- Si se desea escribir la solución óptima además del valor óptimo U de la función objetivo:
 - Cada vez que se añade un nodo a la cola, hay que guardar en una tabla auxiliar de nodos padre el nodo añadido junto a su padre o bien **guardar con cada nodo una identificación de su padre**



Un problema de planificación de tareas a plazo fijo

- Se tiene un conjunto $C = \{1, 2, \dots, n\}$ de tareas y en cada instante sólo se puede realizar una tarea
- Cada tarea, $1 \leq i \leq n$, tiene asociada una terna (t_i, d_i, w_i)
 - t_i es la **duración** de la tarea i
 - la tarea i debe terminarse antes del **plazo** d_i
 - hay una **penalización** w_i si la tarea i no termina antes de su plazo
- Se quiere encontrar un subconjunto $S \subseteq C$ tal que todas las tareas de S puedan ser ejecutadas antes de sus plazos y la penalización total sea mínima.

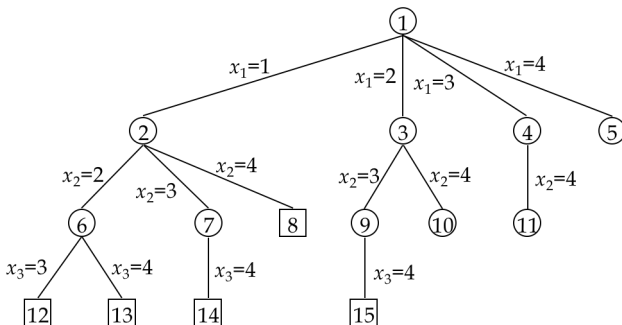


Ejemplo

- $n = 4$
 - $(t_1, d_1, w_1) = (1, 1, 5)$
 - $(t_2, d_2, w_2) = (2, 3, 10)$
 - $(t_3, d_3, w_3) = (1, 2, 6)$
 - $(t_4, d_4, w_4) = (1, 1, 3)$
-
- Espacio de estados: todos los subconjuntos de $\{1, 2, 3, 4\}$
 - Dos representaciones posibles:
 - Tuplas de tamaño variable
 - Tuplas de tamaño fijo



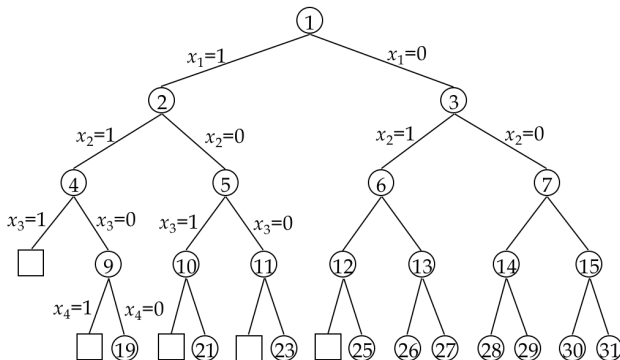
Representación con tuplas de tamaño variable



- Nodos cuadrados: estados no factibles
- Nodos circulares: soluciones factibles
- Nodo 9: solución óptima (es la única):
 - $S = \{2, 3\}$; penalización total = 8



Representación con tuplas de tamaño fijo



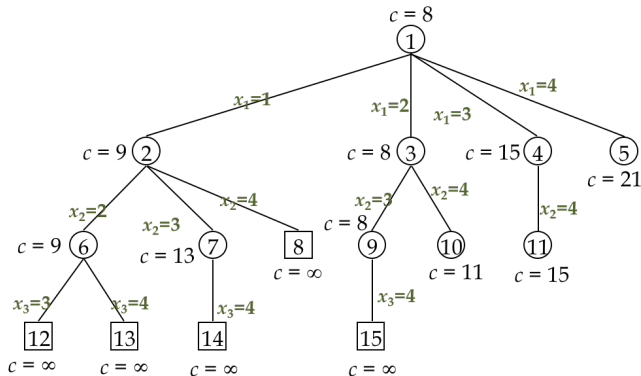
- Nodos cuadrados: estados no factibles
- Hojas circulares: soluciones factibles
- Nodo 25 solución óptima: $S = \{2, 3\}$; penalización total = 8



Definición de la función de coste $c(x)$

Para todo nodo circular x con algún hijo circular $c(x)$ es la mínima penalización correspondiente a todos los nodos descendientes de x . Para todo nodo cuadrado x : $c(x) = \infty$.

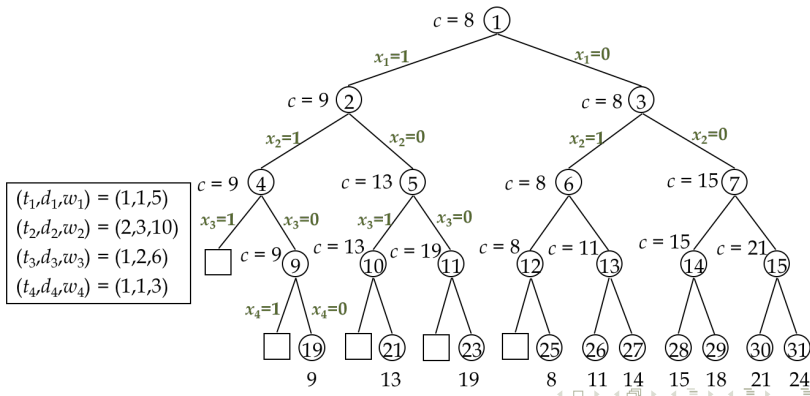
$(t_1, d_1, w_1) = (1, 1, 5)$
 $(t_2, d_2, w_2) = (2, 3, 10)$
 $(t_3, d_3, w_3) = (1, 2, 6)$
 $(t_4, d_4, w_4) = (1, 1, 3)$





Definición de la función de coste $c(x)$

Para todo nodo circular x con algún hijo circular $c(x)$ es la mínima penalización correspondiente a todos los nodos descendientes de x . Para todo nodo cuadrado x : $c(x) = \infty$.





Definición de la función de estimación $\hat{c}(x) \leq c(x)$

Sean:

- $S_x = \{\text{tareas seleccionadas para } S \text{ hasta el nodo } x\}$
- $m_x = \max\{i \mid i \in S_x\}$

Entonces:

Para los nodos circulares:

$$\hat{c}(x) = \sum_{i < m, i \notin S_x} w_i$$

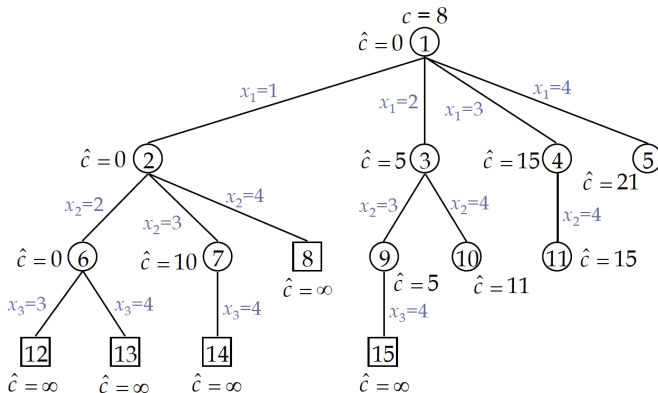
Penalización debida a las tareas que ya han quedado excluidas de x .

Para los nodos cuadrados:

$$\hat{c}(x) = \infty$$



Definición de la función de estimación $\hat{c}(x) \leq c(x)$

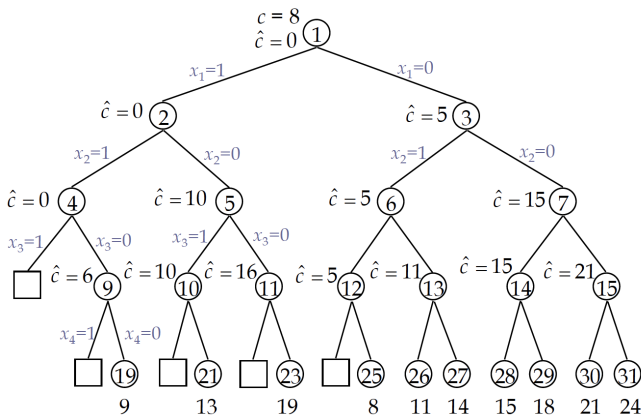


$$\hat{c}(x) = \sum_{\substack{i < m_x \\ i \notin S_x}} w_i$$

$$\begin{aligned} (t_1, d_1, w_1) &= (1, 1, 5) \\ (t_2, d_2, w_2) &= (2, 3, 10) \\ (t_3, d_3, w_3) &= (1, 2, 6) \\ (t_4, d_4, w_4) &= (1, 1, 3) \end{aligned}$$



Definición de la función de estimación $\hat{c}(x) \leq c(x)$



$$\hat{c}(x) = \sum_{\substack{i < m_x \\ i \notin S_x}} w_i$$

$$\begin{aligned} (t_1, d_1, w_1) &= (1, 1, 5) \\ (t_2, d_2, w_2) &= (2, 3, 10) \\ (t_3, d_3, w_3) &= (1, 2, 6) \\ (t_4, d_4, w_4) &= (1, 1, 3) \end{aligned}$$



Definición de la función de poda $\mathcal{U}$

- (Recordar) $\mathcal{U}$ debe ser cota superior del coste de una solución de mínimo coste
- Para cada nodo x , se puede calcular un valor de $\mathcal{U}$:

$$\mathcal{U}(x) = \sum_{i \notin S_x} w_i$$

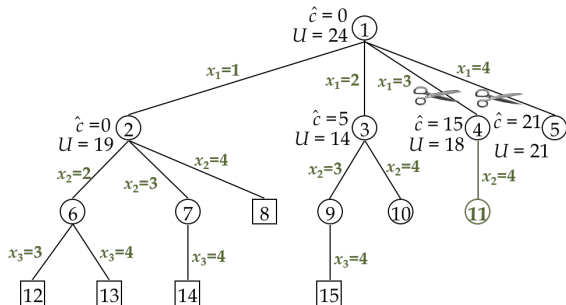


Algoritmo de ramificación y poda

Estrategia de mínimo coste con representación de tamaño variable

$$U = \sum_{i \notin S_x} w_i$$

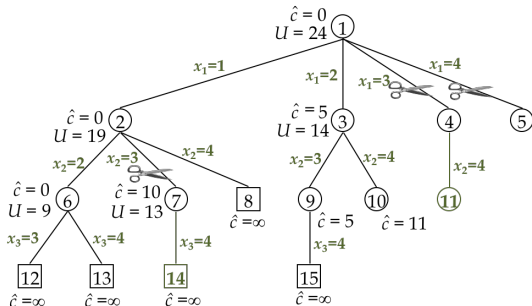
$(t_1, d_1, w_1) = (1, 1, 5)$
 $(t_2, d_2, w_2) = (2, 3, 10)$
 $(t_3, d_3, w_3) = (1, 2, 6)$
 $(t_4, d_4, w_4) = (1, 1, 3)$



- Empieza con $\mathcal{U} = 24$ y ① como nodo en expansión (único nodo vivo)
- Se expande ① generando: ② ③ ④ ⑤
- $\mathcal{U}$ se actualiza a 19 al generar ② (y a 14 al generar ③)
- ④ ⑤ se matan porque $\hat{c}(4) = 15 > \mathcal{U}$, $\hat{c}(5) = 21 > \mathcal{U}$

Estrategia de mínimo coste con representación de tamaño variable

$$\begin{aligned}(t_1, d_1, w_1) &= (1, 1, 5) \\ (t_2, d_2, w_2) &= (2, 3, 10) \\ (t_3, d_3, w_3) &= (1, 2, 6) \\ (t_4, d_4, w_4) &= (1, 1, 3)\end{aligned}$$

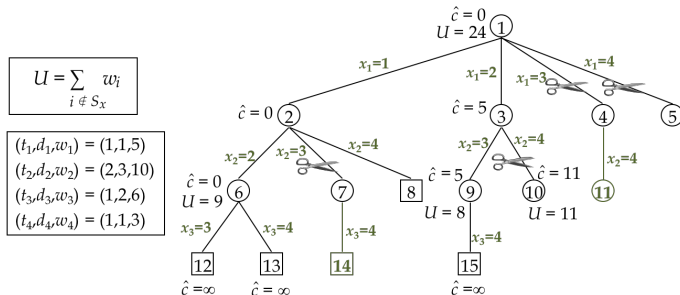


- El siguiente nodo en expansión: ② ($\hat{c}(2) < \hat{c}(3)$)
- Se expande ② generando: ⑥ ⑦ ⑧; $\mathcal{U}$ se actualiza a 9 al generar ⑥
- ⑦ ⑧ se matan porque $\hat{c}(7) = 10 > \mathcal{U} = 9$, $\hat{c}(8) = \infty$
- Los nodos vivos son: ③ ⑥



Algoritmo de ramificación y poda

Estrategia de mínimo coste con representación de tamaño variable

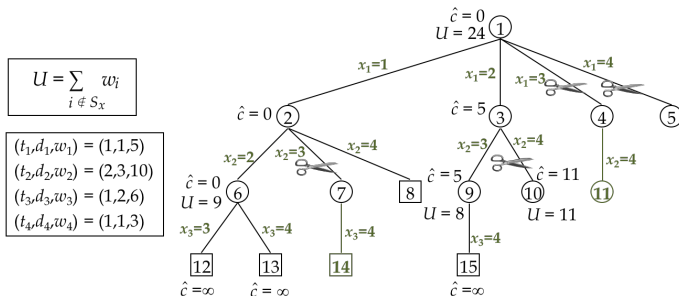


- El siguiente nodo en expansión: ⑥ ($\hat{c}(6) < \hat{c}(3)$)
- Se expande ⑥ generando: ⑫ ⑬ no factibles
- El siguiente nodo en expansión: ③
- Se expande ③ generando: ⑨ ⑩; $\mathcal{U}$ se actualiza a 8 al generar ⑨
- ⑩ se mata porque $\hat{c}(10) = 11 > \mathcal{U} = 8$



Algoritmo de ramificación y poda

Estrategia de mínimo coste con representación de tamaño variable



- El único nodo vivo es (9): se convierte en el nodo en expansión
- Su único hijo es no factible, por tanto se acaba la búsqueda
- La solución es el nodo (9)



Problema de la mochila 0-1

- Hay n objetos y una mochila
- El objeto i tiene peso $p_i > 0$ y su inclusión en la mochila produce un beneficio $b_i > 0$
- **Objetivo:** llenar la mochila de capacidad $C > 0$, de manera que se maximice el beneficio

$$\begin{aligned} \max \quad & \sum_{i=1}^n b_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n p_i x_i \leq C \\ & x_i \in \{0, 1\}, \quad 1 \leq i \leq n \end{aligned}$$



Problema de la mochila 0-1

- Hay n objetos y una mochila
- El objeto i tiene peso $p_i > 0$ y su inclusión en la mochila produce un beneficio $b_i > 0$
- **Objetivo:** llenar la mochila de capacidad $C > 0$, de manera que se maximice el beneficio

$$\begin{aligned} \max \quad & \sum_{i=1}^n b_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n p_i x_i \leq C \\ & x_i \in \{0, 1\}, \quad 1 \leq i \leq n \end{aligned}$$

¿Podemos plantear una solución de ramificación y poda de mínimo coste?



Transformación del problema

Problema de mínimo

- Cambiamos signo a la función de beneficio ...

$$\begin{aligned} \text{min} \quad & - \sum_{i=1}^n b_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n p_i x_i \leq C \\ & x_i \in \{0, 1\}, \quad 1 \leq i \leq n \end{aligned}$$



Definición de la función de coste

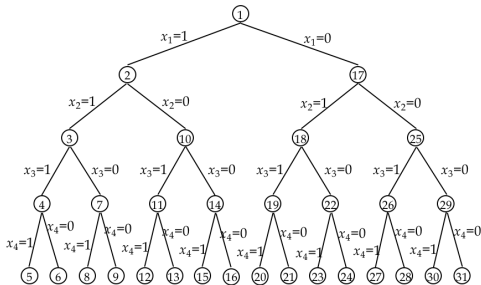
Representación tuplas de tamaño fijo

- Las hojas x tales que:

$$\sum_{i=1}^n p_i x_i \leq C$$

representan soluciones **factibles**

- El resto de las hojas representan soluciones **no factibles**



$$c(x) = \begin{cases} -\sum_{i=1}^n b_i x_i, \\ \infty \\ \min(c(x_{izdo}), c(x_{dcho})) \end{cases}$$

si x solución factible

si x solución no factible

si x no hoja, x_{izdo} , x_{dcho} sus hijos



Definición de las funciones de estimación y poda

Representación tuplas de tamaño fijo

- Necesitamos dos funciones $\hat{c}$ y $\mathcal{U}$ tales que: $\hat{c}(x) \leq c(x) \leq \mathcal{U}(x)$, para cada nodo x



Definición de las funciones de estimación y poda

Representación tuplas de tamaño fijo

- Necesitamos dos funciones $\hat{c}$ y $\mathcal{U}$ tales que: $\hat{c}(x) \leq c(x) \leq \mathcal{U}(x)$, para cada nodo x
- Definición sencilla de $\mathcal{U} \Rightarrow$ Beneficio actual cambiado de signo



Definición de las funciones de estimación y poda

Representación tuplas de tamaño fijo

- Necesitamos dos funciones $\hat{c}$ y $\mathcal{U}$ tales que: $\hat{c}(x) \leq c(x) \leq \mathcal{U}(x)$, para cada nodo x
- Definición sencilla de $\mathcal{U} \Rightarrow$ **Beneficio actual cambiado de signo**
 - Si x es un nodo de nivel i , con $1 \leq i \leq n$, se han asignado ya valores a x_k , $k \leq i$, por tanto:

$$c(x) \leq - \sum_{k=1}^i b_k x_k = \mathcal{U}_{sencillo}(x)$$



Definición de las funciones de estimación y poda

Representación tuplas de tamaño fijo

- Necesitamos dos funciones $\hat{c}$ y $\mathcal{U}$ tales que: $\hat{c}(x) \leq c(x) \leq \mathcal{U}(x)$, para cada nodo x
- Definición sencilla de $\mathcal{U} \Rightarrow$ **Beneficio actual cambiado de signo**
 - Si x es un nodo de nivel i , con $1 \leq i \leq n$, se han asignado ya valores a x_k , $k \leq i$, por tanto:

$$c(x) \leq - \sum_{k=1}^i b_k x_k = \mathcal{U}_{sencillo}(x)$$

- La función de poda se puede mejorar: $\mathcal{U}_{mejor}(x) \leq \mathcal{U}_{sencillo}(x)$



Función de poda mejorada

Datos del problema

- Definimos el tipo de dato **Mochila**: $\{b, p, C, n\}$
 - b (beneficios) es una tupla de n reales (positivos)
 - p (pesos) es una tupla de n reales (positivos)
 - C (capacidad) es un número real (positivo)
 - n es el número de objetos
- Los objetos de una *mochila* están ordenados según el beneficio unitario:

$$\forall i = 1, \dots, mochila.n-1 : \frac{mochila.b[i]}{mochila.p[i]} \geq \frac{mochila.b[i+1]}{mochila.p[i+1]}$$



Algoritmo de la función de poda mejorada

```
1  // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2  //      peso_actual número real (capacidad ya ocupada de la mochila),
3  //      Usencillo número real (valor de U calculado de forma fácil),
4  //      i número entero (índice del primer objeto a considerar),
5  double U_mejorada(Mochila mochila, double peso_actual,
6                   double Usencillo, int i) {
7      Umejor = Usencillo;
8      peso = peso_actual;
9      for each  $j \in \{i, \dots, mochila.n\}$  {
10         if peso+mochila.p[i] <= mochila.C {
11             peso = peso + mochila.p[i];
12             Umejor = Umejor - mochila.b[i];
13         }
14     }
15     return Umejor;
16 }
```



Algoritmo de la función de poda mejorada

```

1  // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2  //      peso_actual número real (capacidad ya ocupada de la mochila),
3  //      Usencillo número real (valor de U calculado de forma fácil),
4  //      i número entero (índice del primer objeto a considerar),
5  double U_mejorada(Mochila mochila, double peso_actual,
6                    double Usencillo, int i) {
7      Umejor = Usencillo;
8      peso = peso_actual;
9      for each  $j \in \{i, \dots, mochila.n\}$  {
10         if peso+mochila.p[i] <= mochila.C {
11             peso = peso + mochila.p[i];
12             Umejor = Umejor - mochila.b[i];
13         }
14     }
15     return Umejor;
16 }

```

- peso_actual: $\sum_{k=1}^i mochila.p[k]x_k$
- Usencillo: $-\sum_{k=1}^i mochila.b[k]x_k$, nivel: $i + 1$



Definición de la función de estimación

Representación tuplas de tamaño fijo

- Reutilizamos la función de poda que se definió para el método de búsqueda con retroceso:
 - Era una cota superior del valor de la mejor solución posible al expandir el nodo y sus descendientes
 - Ahora hemos cambiado de signo la función objetivo
 - Luego, cambiada de signo, es una cota inferior de $c(x)$



Definición de la función de estimación

Representación tuplas de tamaño fijo

- Reutilizamos la función de poda que se definió para el método de búsqueda con retroceso:
 - Era una cota superior del valor de la mejor solución posible al expandir el nodo y sus descendientes
 - Ahora hemos cambiado de signo la función objetivo
 - Luego, cambiada de signo, es una cota inferior de $c(x)$
- ¿Cómo se calculó esa cota?



Definición de la función de estimación

Representación tuplas de tamaño fijo

- Reutilizamos la función de poda que se definió para el método de búsqueda con retroceso:
 - Era una cota superior del valor de la mejor solución posible al expandir el nodo y sus descendientes
 - Ahora hemos cambiado de signo la función objetivo
 - Luego, cambiada de signo, es una cota inferior de $c(x)$
- ¿Cómo se calculó esa cota?
 - En el nodo x de nivel i ya se han determinado $x_k, k \leq i$
 - Se relaja el requisito de integridad para los siguientes elementos $x_{i+1}, \dots, x_n$
 - Es decir, estos elementos pueden tomar valores reales entre 0 y 1
 - Se aplica el algoritmo voraz ...



Función de estimación del coste

```
1  // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2  //      libre es la capacidad aún libre de la mochila,
3  //      ben es el beneficio actual,
4  //      i es el índice del primer objeto a considerar
5
6  double cota(Mochila mochila, double libre, double ben, int i) {
7      if i>mochila.n or libre=0.0 { return ben; }
8      else {
9          if mochila.p[i] > libre
10             { return ben + libre/mochila.p[i]*mochila.b[i]; }
11         else
12             { return cota(mochila, libre-mochila.p[i], ben+mochila.b[i], i+1); }
13     }
14 }
```



Función de estimación del coste

```

1  // Pre: en mochila, los objetos están ordenados según el beneficio unitario
2  //      libre es la capacidad aún libre de la mochila,
3  //      ben es el beneficio actual,
4  //      i es el índice del primer objeto a considerar
5
6  double cota(Mochila mochila, double libre, double ben, int i) {
7      if i>mochila.n or libre=0.0 { return ben; }
8      else {
9          if mochila.p[i] > libre
10             { return ben + libre/mochila.p[i]*mochila.b[i]; }
11         else
12             { return cota(mochila, libre-mochila.p[i], ben+mochila.b[i], i+1); }
13     }
14 }

```

$\hat{c}(x) = -\text{cota}(\text{mochila}, \text{libre}, \text{ben}, i+1)$ donde:

- $\text{libre} = \text{mochila}.C - \sum_{k=1}^i \text{mochila}.p[k]x_k$,
- $\text{ben} = \sum_{k=1}^i \text{mochila}.b[k]x_k$



Ejemplo ../..

Instancia del problema $pb = \{b, p, C, n\}$:

- $b = [10, 10, 12, 18]$, $p = [2, 4, 6, 9]$, $C = 15$, $n = 4$
- Los objetos están ordenados según el beneficio unitario (de mayor a menor)



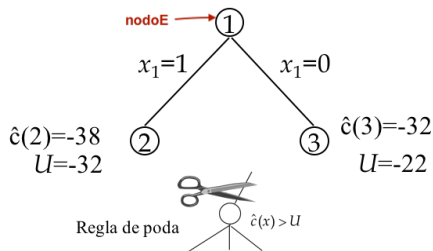
Ejemplo ../..

Instancia del problema $pb = \{b, p, C, n\}$:

- $b = [10, 10, 12, 18]$, $p = [2, 4, 6, 9]$, $C = 15$, $n = 4$
- Los objetos están ordenados según el beneficio unitario (de mayor a menor)
- Nodo en expansión: ① (raíz, nivel 0, ninguna x_i asignada)

$$\begin{aligned}
 \hat{c}(1) &= -\text{cota}(pb, 15, 0, 1) \\
 &= -\text{cota}(pb, 13, 10, 2) \\
 &= -\text{cota}(pb, 9, 20, 3) \\
 &= -\text{cota}(pb, 3, 32, 4) \\
 &= -(32 + 3/9 * 18) = -38
 \end{aligned}$$

$$U_{\text{mejor}}(1) = -32 \Rightarrow U \text{ cota inicial}$$



Ejemplo ../..

$$b = [10, 10, 12, 18]$$

$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$



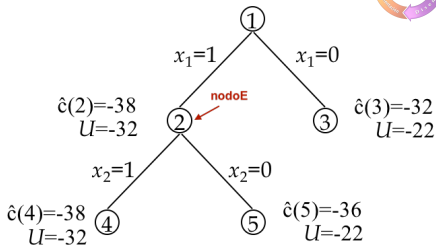
Ejemplo ../..

$$b = [10, 10, 12, 18]$$

$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$

- Nodos vivos: ② ③
- ¿Siguiente nodo?



Ejemplo ../..

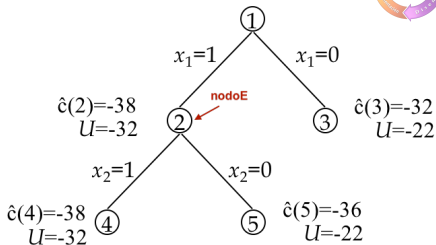
$$b = [10, 10, 12, 18]$$

$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$

- Nodos vivos: ② ③
- ¿Siguiente nodo?

- Nodos vivos: ③ ④ ⑤
- ¿Siguiente nodo?



Ejemplo ../..

$$b = [10, 10, 12, 18]$$

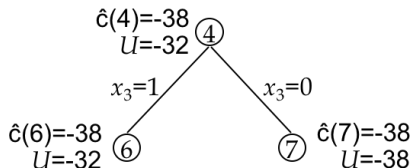
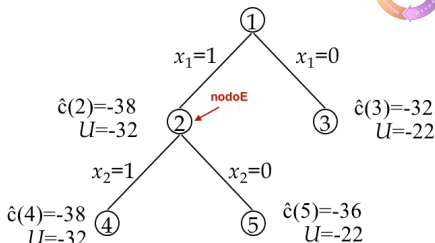
$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$

- Nodos vivos: ② ③
- ¿Siguiente nodo?

- Nodos vivos: ③ ④ ⑤
- ¿Siguiente nodo?

- Nodos vivos: ③ ⑤ ⑥ ⑦ ($U = -38$)
- ¿Siguiente nodo?





Ejemplo

- Si se elige ⑥ el hijo izdo se elimina por no ser factible, el dcho ⑥' se poda (coste $-32 > \mathcal{U}$)

$$b = [10, 10, 12, 18]$$

$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$



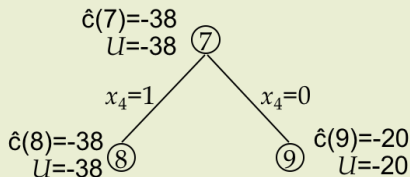
Ejemplo

- Si se elige ⑥ el hijo izdo se elimina por no ser factible, el dcho ⑥' se poda (coste $-32 > \mathcal{U}$)
- Nodos vivos: ③ ⑤ ⑦
- Siguiente nodo ⑦
- El nodo ⑨ no se añade a la lista (poda $\hat{c}(9) > \mathcal{U}$)

$$b = [10, 10, 12, 18]$$

$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$





Ejemplo

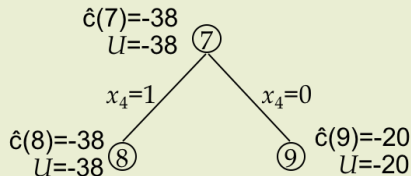
- Si se elige ⑥ el hijo izdo se elimina por no ser factible, el dcho ⑥' se poda (coste $-32 > \mathcal{U}$)

$$b = [10, 10, 12, 18]$$

$$p = [2, 4, 6, 9]$$

$$C = 15, n = 4$$

- Nodos vivos: ③ ⑤ ⑦
- Siguiente nodo ⑦
- El nodo ⑨ no se añade a la lista (poda $\hat{c}(9) > \mathcal{U}$)



- Nodos vivos: ③ ⑤ ⑧
- Siguiente nodo? ⑧ es hoja: el valor óptimo es 38 (nuevo valor de $\mathcal{U}$)
- Los demás nodos en la lista: $\hat{c}(x) > \mathcal{U}$



Observaciones

Cabe pensar en una versión con estrategia ciega de ramificación (FIFO, por ejemplo):

- Por un lado la intuición nos dice que la estrategia de mínimo coste se acerca más deprisa a la solución óptima
- Por otra parte, la inserción y borrado en la cola con prioridades de nodos vivos tiene coste logarítmico, mientras que con una cola FIFO, el coste es constante . . .



El problema del viajante de comercio

Encontrar un recorrido de longitud mínima para un viajante que tiene que visitar varias ciudades y volver al punto de partida, conocida la distancia existente entre cada dos ciudades.

- Es decir, dado un grafo dirigido con arcos de longitud no negativa, se trata de encontrar un circuito de longitud mínima que comience y termine en el mismo vértice y pase exactamente una vez por cada uno de los vértices restantes (circuito *hamiltoniano*).



Soluciones propuestas

El problema del viajante de comercio es probablemente el problema NP-completo más estudiado en términos de soluciones propuestas^a

^aU. Manber: Introduction to Algorithms. A Creative Approach. Addison Wesley, 1989.

- Solución heurística voraz
 - Calcula una solución sub-óptima
- Programación dinámica:
 - Coste en tiempo: $O(n^2 2^n)$



Soluciones propuestas

El problema del viajante de comercio es probablemente el problema NP-completo más estudiado en términos de soluciones propuestas^a

^aU. Manber: Introduction to Algorithms. A Creative Approach. Addison Wesley, 1989.

- Solución heurística voraz
 - Calcula una solución sub-óptima
- Programación dinámica:
 - Coste en tiempo: $O(n^2 2^n)$
- El coste del caso peor de la solución que vamos a ver seguirá estando en $O(n^2 2^n)$
 - Sin embargo, el uso de buenas funciones de poda mejora la eficiencia en muchos casos con respecto a la solución de programación dinámica



Formalización del problema

Sean $G = (V, A)$ un grafo orientado:

- n vértices $V = \{1, 2, \dots, n\}$
 - L_{ij} la longitud de $(i, j) \in A$. Si el arco (i, j) no existe: $L_{ij} = \infty$
 - El circuito buscado empieza en el vértice 1
-
- Espacio de búsqueda: $E = \{1, \pi, 1 \mid \pi \text{ es una permutación de } (2, 3, \dots, n)\}$
 - Tamaño: $|E| = (n - 1)!$

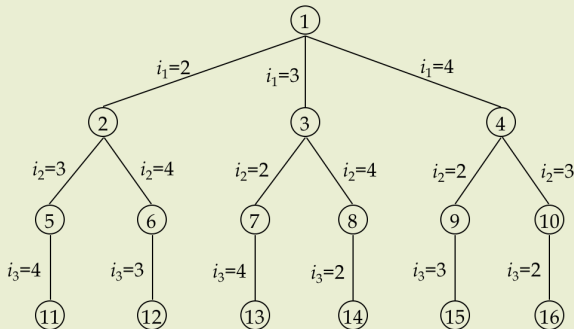
Reducción del espacio de búsqueda

$E = \{1, \pi, 1 \mid \pi = i_1, i_2, \dots, i_{n-1}, \text{ es una permutación de } (2, 3, \dots, n) \text{ tal que } (i_j, i_{j+1}) \in A, 0 \leq j \leq n - 1 \text{ y } i_0 = i_n = 1\}$.



Representación del espacio de estados

Caso de un grafo completo $|V| = 4$



Cada hoja es una solución y representa el viaje definido por el camino desde la raíz hasta la hoja.



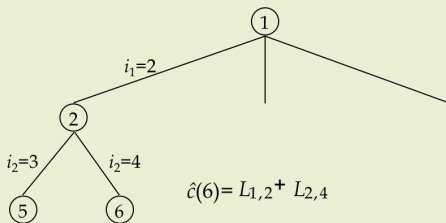
Definición de la función de coste

- La función c debe ser tal que un nodo solución x con valor mínimo de $c(x)$ corresponda a un recorrido de longitud mínima
- Por ejemplo:
 - Si x es una hoja, entonces $c(x)$ es la longitud del hamiltoniano definido por el camino desde la raíz hasta x
 - Si x no es una hoja, entonces $c(x)$ es el coste de una hoja de coste mínimo del sub-árbol con raíz x



Definición de la función de estimación

- La función $\hat{c}$ debe ser tal que $\hat{c}(x) \leq c(x)$, para cada nodo x
- Una función muy fácil:
 - $\hat{c}(x)$ es la distancia total del recorrido definido por el camino desde la raíz hasta x



- Si es una hoja, es obvio que $\hat{c}(x) = c(x)$



Definición de la función de estimación

Mejora de $\hat{c}(x)$

Matriz de distancias reducidas

- Una fila (columna) de la matriz de distancias se dice **reducida** si sus elementos son no negativos y contiene al menos un 0 (cero)
- Una matriz de distancias se dice reducida si cada fila y columna es reducida



Definición de la función de estimación

Mejora de $\hat{c}(x)$

Matriz de distancias reducidas

- Una fila (columna) de la matriz de distancias se dice **reducida** si sus elementos son no negativos y contiene al menos un 0 (cero)
- Una matriz de distancias se dice reducida si cada fila y columna es reducida
- Para cada vértice $k, 1 \leq k \leq n$, todo circuito hamiltoniano incluye exactamente un arco de la forma $(k, \cdot)$ y exactamente un arco de la forma $(\cdot, k)$
- Si se resta una constante t de cada elemento de una fila (columna) de la matriz de distancia, la longitud de todo hamiltoniano se reduce exactamente en t y un hamiltoniano de distancia mínima lo sigue siendo



Definición de la función de estimación

Ejemplo de reducción

- Reducción fila 1, $t = 10$

∞	20	30	10	11
15	∞	16	4	2
3	5	∞	2	4
19	6	18	∞	3
16	4	7	16	∞

- Se repite el proceso hasta obtener la matriz de distancia reducida

- Reducción: $L = 25$

∞	20	30	10	11
15	∞	16	4	2
3	5	∞	2	4
19	6	18	∞	3
16	4	7	16	∞

La cantidad L total restada es una cota inferior de la longitud de un hamiltoniano de longitud mínima, luego sirve como estimación de $\hat{c}(x)$ para el nodo x raíz.



Definición de la función de estimación

Cálculo de $\hat{c}(x)$ para los nodos x intermedios (no raíz y no hoja)

- Sea A la matriz de distancia reducida para el nodo y
- Sea x un hijo de y que corresponda a incluir el arco (i, j) en el recorrido y que no sea hoja
- La matriz B reducida para x se calcula de la siguiente forma:



Definición de la función de estimación

Cálculo de $\hat{c}(x)$ para los nodos x intermedios (no raíz y no hoja)

- Sea A la matriz de distancia reducida para el nodo y
 - Sea x un hijo de y que corresponda a incluir el arco (i, j) en el recorrido y que no sea hoja
 - La matriz B reducida para x se calcula de la siguiente forma:
 - 1 Cambiar todos los elementos de la fila i y de la columna j de A por ∞
 - 2 Cambiar el elemento $A(j, 1)$ por ∞ para evitar considerar el arco $(j, 1)$
 - 3 B es la matriz que se obtiene al reducir todas las filas y columnas de la matriz resultante (excepto aquéllas formadas sólo por ∞).
- Si r es el valor total restado:

$$\hat{c}(x) = \hat{c}(y) + A(i, j) + r$$



Definición de la función de poda

- Una función de poda trivial:
 - Inicialmente, $\mathcal{U} = \infty$
 - Se modifica $\mathcal{U}$ sólo cuando se llega a un nodo x que es hoja (solución factible), entonces:

$$U = \min\{\mathcal{U}, c(x)\}$$

- Se puede pensar en mejores funciones de poda



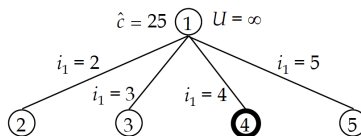
Ejemplo

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

Grafo original

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

Reducción: $L = 25$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & 2 & 0 \\ 0 & \infty & \infty & 0 & 2 \\ 15 & \infty & 12 & \infty & 0 \\ 11 & \infty & 0 & 12 & \infty \end{bmatrix}$$


$$\hat{c} = 25 + 10 = 35 \quad \hat{c} = 25 + 17 + 11 = 53 \quad \hat{c} = 25 \quad \hat{c} = 25 + 1 + 5 = 31$$

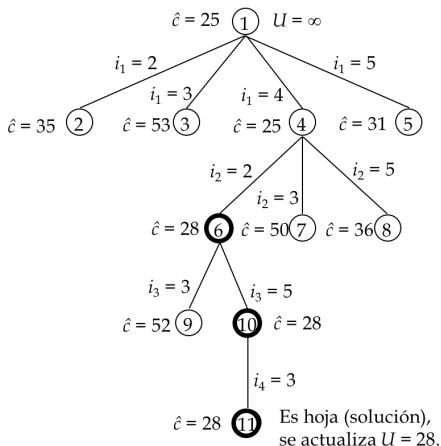
$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 10 & \infty & 9 & 0 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 12 & 0 & 9 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix}$$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 1 & \infty & \infty & 2 & 0 \\ \infty & 3 & \infty & 0 & 2 \\ 4 & 3 & \infty & \infty & 0 \\ 0 & 0 & \infty & 12 & \infty \end{bmatrix}$$

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$



Ejemplo



- El siguiente nodo a expandir sería ⑤ pero $\hat{c}(5) > U$ luego el algoritmo termina
- El hamiltoniano mínimo es 1,4,2,5,3,1



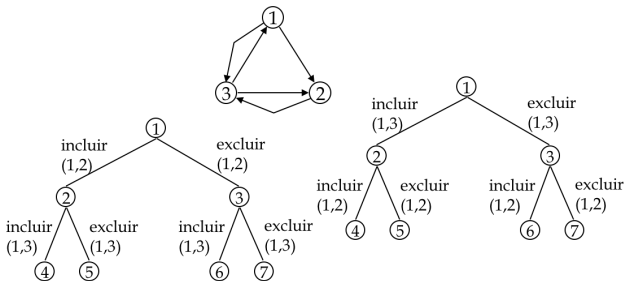
Otras soluciones

- Un hamiltoniano es un **conjunto** de n arcos
- Además, para cada vértice i , $1 \leq i \leq n$, debe haber en ese conjunto exactamente un arco de la forma $(i, \cdot)$ y uno de la forma $(\cdot, i)$
- Representación del espacio de estados como un **árbol binario**
 - Un hijo izquierdo representa la **inclusión** de un arco en el hamiltoniano
 - Un hijo derecho representa la **exclusión** de ese arco



Representaciones con árboles binarios

- Hay distintos árboles dependiendo de qué arco se elige para incluir/excluir en cada paso



- ¿Cómo elegir el arco por el que empezar?
 - Depende de los datos del problema
- Árbol de estados **dinámico**: el método de construcción depende de los datos del problema



Representaciones con árboles binarios

- Si se elige, para empezar, el arco (i, j)
 - El subárbol izquierdo representa todos los recorridos que incluyen (i, j)



Representaciones con árboles binarios

- Si se elige, para empezar, el arco (i, j)
 - El subárbol izquierdo representa todos los recorridos que incluyen (i, j)
 - El subárbol derecho representa todos los recorridos que no lo incluyen



Representaciones con árboles binarios

- Si se elige, para empezar, el arco (i, j)
 - El subárbol izquierdo representa todos los recorridos que incluyen (i, j)
 - El subárbol derecho representa todos los recorridos que no lo incluyen
 - Si hay un recorrido óptimo incluido en el subárbol izquierdo, entonces sólo faltan por seleccionar $n-1$ arcos para encontrarlo



Representaciones con árboles binarios

- Si se elige, para empezar, el arco (i, j)
 - El subárbol izquierdo representa todos los recorridos que incluyen (i, j)
 - El subárbol derecho representa todos los recorridos que no lo incluyen
 - Si hay un recorrido óptimo incluido en el subárbol izquierdo, entonces sólo faltan por seleccionar $n-1$ arcos para encontrarlo
 - Mientras que si todos los recorridos óptimos están en el derecho, hay que seleccionar todavía n arcos



Representaciones con árboles binarios

- Si se elige, para empezar, el arco (i, j)
 - El subárbol izquierdo representa todos los recorridos que incluyen (i, j)
 - El subárbol derecho representa todos los recorridos que no lo incluyen
 - Si hay un recorrido óptimo incluido en el subárbol izquierdo, entonces sólo faltan por seleccionar $n-1$ arcos para encontrarlo
 - Mientras que si todos los recorridos óptimos están en el derecho, hay que seleccionar todavía n arcos
- Hay que intentar seleccionar un arco que tenga **la máxima probabilidad de estar en el recorrido óptimo**



Representaciones con árboles binarios

- Si se elige, para empezar, el arco (i, j)
 - El subárbol izquierdo representa todos los recorridos que incluyen (i, j)
 - El subárbol derecho representa todos los recorridos que no lo incluyen
 - Si hay un recorrido óptimo incluido en el subárbol izquierdo, entonces sólo faltan por seleccionar $n-1$ arcos para encontrarlo
 - Mientras que si todos los recorridos óptimos están en el derecho, hay que seleccionar todavía n arcos
- Hay que intentar seleccionar un arco que tenga **la máxima probabilidad de estar en el recorrido óptimo**
- Varias heurísticas posibles:
 - Elegir un arco que produzca un subárbol derecho cuya raíz tenga $\hat{c}(x)$ máxima
 - Elegir un arco tal que la diferencia entre $\hat{c}(x_{\text{izquierdo}})$ y $\hat{c}(x_{\text{derecho}})$ sea máxima
 - ...

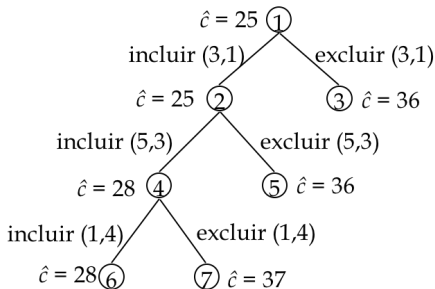


Representaciones con árboles binarios

Heurística del máximo coste estimado

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

Grafo original

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$
Reducción inicial: $L = 25$ 

- Al nodo ⑥ se han elegido ya tres arcos: (3, 1), (5, 3) y (1, 4)
- Para los restantes, sólo queda ya una opción: (4, 2) y (2, 5)
- Hamiltoniano: 5,3,1,4,2,5 con distancia total: 28 (así, $\mathcal{U} = 28$)
- El siguiente nodo a expandir es ③ con $\hat{c}(3) > \mathcal{U}$. El algoritmo acaba.



Observaciones

- En el último ejemplo, el método de ramificación y poda se usa sólo para subárboles grandes
- Una vez que se llega a subárboles pequeños (4 ó 6 nodos) es más eficiente construirlos enteros y calcular el valor exacto de la función de coste

No hay forma de calcular analíticamente cuál de las soluciones del problema del viajante de comercio es más eficiente.



Consideraciones finales sobre eficiencia

- ¿Es posible disminuir en algún caso el número de nodos generados mediante la expansión de nodos x con $\hat{c}(x) > \mathcal{U}$?

Nunca porque el valor de $\mathcal{U}$ no será modificado (reducido) si se expande tal nodo x



Consideraciones finales sobre eficiencia

- ¿Es posible disminuir en algún caso el número de nodos generados mediante la expansión de nodos x con $\hat{c}(x) > \mathcal{U}$?

Nunca porque el valor de $\mathcal{U}$ no será modificado (reducido) si se expande tal nodo x

- Si hay dos funciones de poda $\mathcal{U}_1$, $\mathcal{U}_2$ diferentes y $\mathcal{U}_1 < \mathcal{U}_2$ entonces **siempre** es mejor usar $\mathcal{U}_1$ (nunca dará lugar a más nodos)



Consideraciones finales sobre eficiencia

- ¿Es posible disminuir en algún caso el número de nodos generados mediante la expansión de nodos x con $\hat{c}(x) > \mathcal{U}$?

Nunca porque el valor de $\mathcal{U}$ no será modificado (reducido) si se expande tal nodo x

- Si hay dos funciones de poda $\mathcal{U}_1, \mathcal{U}_2$ diferentes y $\mathcal{U}_1 < \mathcal{U}_2$ entonces **siempre** es mejor usar $\mathcal{U}_1$ (nunca dará lugar a más nodos)
- ¿Si se usa una función de estimación mejor, decrece el número de nodos a expandir? ($\hat{c}_2$ es mejor que $\hat{c}_1$ si $\hat{c}_1 \leq \hat{c}_2 \leq c$)



Consideraciones finales sobre eficiencia

- ¿Es posible disminuir en algún caso el número de nodos generados mediante la expansión de nodos x con $\hat{c}(x) > \mathcal{U}$?

Nunca porque el valor de $\mathcal{U}$ no será modificado (reducido) si se expande tal nodo x

- Si hay dos funciones de poda $\mathcal{U}_1, \mathcal{U}_2$ diferentes y $\mathcal{U}_1 < \mathcal{U}_2$ entonces **siempre** es mejor usar $\mathcal{U}_1$ (nunca dará lugar a más nodos)
- ¿Si se usa una función de estimación mejor, decrece el número de nodos a expandir? ($\hat{c}_2$ es mejor que $\hat{c}_1$ si $\hat{c}_1 \leq \hat{c}_2 \leq c$)

No necesariamente. Si se usa una función de estimación mejor con la estrategia de ramificación de **mínimo coste**, el número de nodos generados puede crecer.